

COMPUTER SCIENCE TRIPOS Part IB – 2021 – Paper 4

4 Compiler Construction (tgg22)

Context-free
grammars

- (a) Show that the following grammar for the language of balanced parenthesis is ambiguous.

$$\begin{array}{lcl} S & \rightarrow & \epsilon \\ & | & (S) \\ & | & SS \end{array}$$

[5 marks]

Answer: We need to show two different parse trees associated with one member of the language generated by this grammar. Consider $w = ()()()$. Then the two derivations

$$S \Rightarrow SS \Rightarrow (S)S \Rightarrow ()S \Rightarrow ()SS \Rightarrow ()(S)S \Rightarrow ()()S \Rightarrow ()()(S) \Rightarrow ()()()$$

and

$$S \Rightarrow SS \Rightarrow S(S) \Rightarrow S() \Rightarrow SS() \Rightarrow (S)S() \Rightarrow ()S() \Rightarrow ()(S)() \Rightarrow ()()()$$

are associated with distinct parse trees.

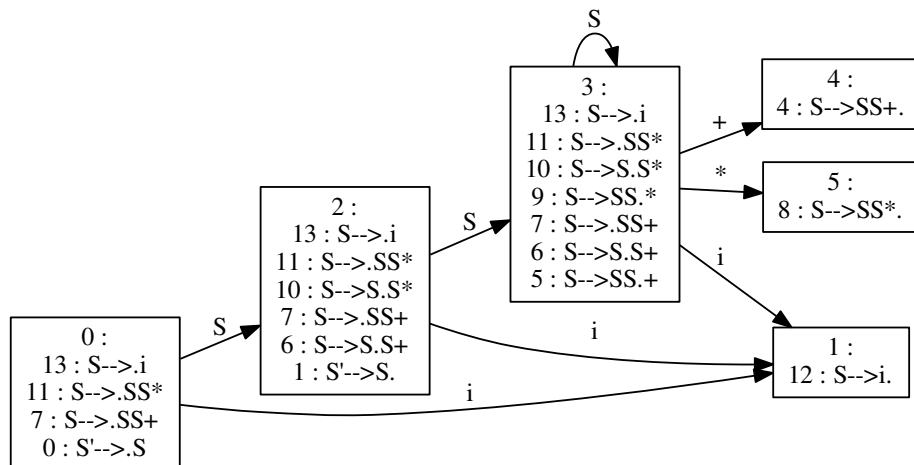
Context-free
grammars

- (b) An arithmetic expression such as $x * ((z * u) + y)$ can be represented without parenthesis in postfix notation as $xzu * y + *$. This representation is ideal for evaluation using a stack machine.

Below is a simple grammar for postfix expressions:

$$\begin{array}{lcl} S & \rightarrow & i \\ & | & SS+ \\ & | & SS* \end{array}$$

The terminal i represents the lexical class of identifiers. Here is a DFA for the LR(0) items of this grammar.



— *Solution notes* —

- (i) Using the DFA, construct the SLR(1) *ACTION* and *GOTO* tables for this grammar. Explain your work. [6 marks]

Answer: Recall that The *ACTION* table is computed as

If $A \rightarrow \alpha \bullet a\beta \in I_i$ and $\delta'(I_i, a) = I_j$
then $ACTION[i, a] = \text{shift } j$

If $A \rightarrow \alpha \bullet \in I_i$ and $A \neq S'$
then for all $a \in FOLLOW(A)$
 $ACTION[i, a] = \text{reduce } A \rightarrow \alpha$

The grammar is simple enough to see by inspection that $FOLLOW(S) = \{\$, i, +, *\}$. Then the *ACTION* table can be computed by inspecting the DFA as follows:

$ACTION[0, i]$	$=$	$=$	shift 1
$ACTION[1, \$]$	$=$	$=$	reduce $S \rightarrow i$
$ACTION[1, *]$	$=$	$=$	reduce $S \rightarrow i$
$ACTION[1, i]$	$=$	$=$	reduce $S \rightarrow i$
$ACTION[1, +]$	$=$	$=$	reduce $S \rightarrow i$
$ACTION[2, \$]$	$=$	$=$	accept!
$ACTION[2, i]$	$=$	$=$	shift 1
$ACTION[3, +]$	$=$	$=$	shift 4
$ACTION[3, i]$	$=$	$=$	shift 1
$ACTION[3, *]$	$=$	$=$	shift 5
$ACTION[4, \$]$	$=$	$=$	reduce $S \rightarrow SS+$
$ACTION[4, *]$	$=$	$=$	reduce $S \rightarrow SS+$
$ACTION[4, i]$	$=$	$=$	reduce $S \rightarrow SS+$
$ACTION[4, +]$	$=$	$=$	reduce $S \rightarrow SS+$
$ACTION[5, \$]$	$=$	$=$	reduce $S \rightarrow SS*$
$ACTION[5, *]$	$=$	$=$	reduce $S \rightarrow SS*$
$ACTION[5, i]$	$=$	$=$	reduce $S \rightarrow SS*$
$ACTION[5, +]$	$=$	$=$	reduce $S \rightarrow SS*$

The *GOTO* table can be read off directly from the DFA. For example, $GOTO[0, S] = 2$, $GOTO[2, S] = 3$, and so on.

- (ii) Show a trace of a parsing of $w = iii * i + *$. Justify every step. [9 marks]

Answer:

symbol stack	state stack	input	action
\$	0	$iii * i + * \$$	shift 1
$\$i$	0, 1	$ii * i + * \$$	reduce $S \rightarrow i$
$\$S$	0	$ii * i + * \$$	push $GOTO[0, S] = 2$
$\$S$	0, 2	$ii * i + * \$$	shift 1
$\$Si$	0, 2, 1	$i * i + * \$$	reduce $S \rightarrow i$
$\$SS$	0, 2	$i * i + * \$$	push $GOTO[2, S] = 3$
$\$SS$	0, 2, 3	$i * i + * \$$	shift 1
$\$SSi$	0, 2, 3, 1	$*i + * \$$	reduce $S \rightarrow i$
$\$SSS$	0, 2, 3	$*i + * \$$	push $GOTO[3, S] = 3$
$\$SSS$	0, 2, 3, 3	$*i + * \$$	shift 5
$\$SSS*$	0, 2, 3, 3, 5	$i + * \$$	reduce $S \rightarrow SS*$
$\$SS$	0, 2	$i + * \$$	push $GOTO[2, S] = 3$
$\$SS$	0, 2, 3	$i + * \$$	shift 1
$\$SSi$	0, 2, 3, 1	$+ * \$$	reduce $S \rightarrow i$
$\$SSS$	0, 2, 3	$+ * \$$	push $GOTO[3, S] = 3$
$\$SSS$	0, 2, 3, 3	$+ * \$$	shift 4
$\$SSS+$	0, 2, 3, 3, 4	$* \$$	reduce $S \rightarrow SS+$
$\$SS$	0, 2	$* \$$	push $GOTO[2, S] = 3$
$\$SS$	0, 2, 3	$* \$$	shift 5
$\$SS*$	0, 2, 3, 5	$\$$	reduce $S \rightarrow SS*$
$\$S$	0	$\$$	push $GOTO[0, S] = 2$
$\$S$	0, 2	$\$$	accept!
